PA#2 — Responsive prototype · rubric

CSC13008 · project assignment · teams of up to 3 · 3% of the course · 100 points

Due **Wednesday, November 11, 2026, 22:00**. This is the second milestone of your project: your team turns the three screens of the GradStudio main flow, *services and chat → advice and clash handling → review the appointment and pay the deposit*, into pages, static HTML and CSS or built with React, in your team repository, that work on a phone, a tablet and a laptop. GradStudio is the final project for the whole class; the screens and the list of problems to fix are in the *GradStudio final project brief*. This rubric is the brief; the session 5 slides summarise it.

Use your assistant. Nothing in this milestone asks you to type code yourself: it scores what the pages do in a browser, what you told the assistant, and how you checked what it wrote.

What you build

- **Three pages that cover the main flow of the GradStudio brief**, in the order of that flow. Your team chooses the screens, the layout, the content and the colours, and names the files; `index.html` is the first page. Together the pages must deliver the features the brief gives its three screens, and none of the problems on its *Fix these* list may appear in them.
- **Links between them work.** Every link or button that moves along the main flow opens the next page. On static pages, use relative links to the file (`studio.html`, not `/studio.html`, and not a folder such as `studio/`, which opens a file listing) so the pages also work when opened straight from the folder. Moving to another page is an `<a href>`, or the submit button of a form with `action="next.html"`. A button with `onclick` does nothing when JavaScript is off. With React, use router links, which render as `<a href>`, or the submit button of a form. Hash routes are fine. A plain button that navigates does not count.
- **Static pages, or React.** Either way the data is realistic placeholder data for GradStudio, not lorem ipsum, with at least one long item (a long Vietnamese name, a long title) to show the layout survives it.
- *Static:* content is written into the HTML. JavaScript is not needed; if you add a little (a menu toggle), every page must still show all its content with JavaScript turned off, because that is how we mark static pages. Try it: DevTools → `Ctrl+Shift+P` → *Disable JavaScript*, then reload.
- *React:* also hand in the built `dist/` folder and list the path of each screen in `PROTOTYPE.md`. We serve `dist/` from a local static server with JavaScript on and the outside network blocked, and we open each path directly: a screen that fills itself only after a click on an earlier screen counts as blank. Bundle your CSS and fonts into the build, because a stylesheet or web font from a CDN does not load when we mark. Every other requirement stays: three screens, three widths, real links and buttons, and 0 errors in the W3C checker on the rendered page. Before you hand in: `npm run build`, then `npx vite preview`, turn your network off, and open each path directly at the three widths. We do not run `npm`, so rebuild after your last change. Zip the working folder, not a fresh clone: `dist/` is usually git-ignored, so a clone loses it. No other JavaScript framework.
- **Responsive, mobile first:** usable at **360, 768 and 1280 px** wide, with no horizontal scroll at any of them (session 3).
- **Semantic and valid:** landmarks, one heading outline, real buttons and links, a label on every input (session 4), and **0 errors** in the W3C checker.

- **A CSS framework is allowed** — Tailwind or Bootstrap — and so is plain CSS. Pick one, as session 3 said, and keep it for the semester. Static pages must open without a build step, so load the framework's **CSS file**: a `<link rel="stylesheet">` from a CDN, or the built CSS committed to the repository. Two traps with JavaScript off: Tailwind's CDN is a `<script>` that builds the CSS in the browser, so the page has no styles at all — build the CSS once with the Tailwind CLI (`npx @tailwindcss/cli -i input.css -o tailwind.css`) and commit the output; and Bootstrap's collapsing navbar (`navbar-expand-lg` with a toggler) needs Bootstrap's JavaScript, so on a phone the menu never opens — use `navbar-expand`, which is always open.

What you hand in

One zip of your team repository folder **including its hidden `.git` folder** (leave `node_modules` out). Zip the folder itself: GitHub's *Download ZIP* and `git archive` leave `.git` out. At the repository root:

- the pages — `index.html` and the other two — and your CSS; with React, the source and the built `dist/`
- the project rules file your assistant reads: `CLAUDE.md`, `AGENTS.md`, `.cursor/rules` or `.github/copilot-instructions.md` (session 2)
- `PROTOTYPE.md` — the sections listed below
- `screenshots/` — every page at the three widths, full page, with JavaScript off (React: on): `<page>-360.png`, `<page>-768.png`, `<page>-1280.png`, where `<page>` is the file name without `.html` (`index-360.png`, `studio-768.png`; nine files). With React, `<page>` comes from the path: `/` is `index`, `/review` and `/#/review` are `review`, `/items/12` is `items-12`. In the DevTools device toolbar: set the width, then ⋮ → *Capture full size screenshot*
- `checks/` — the W3C checker's result for every page, by *File upload* (`validator-<page>.png` or `.txt`, same `<page>` names). With React, do not upload `dist/index.html`: it is an empty shell and shows 0 errors even when the page is broken. Open each screen, copy the `<html>` element's outerHTML in DevTools, add `<!DOCTYPE html>` in front, and paste it into *Text input*.
- `brief.md` — what you actually gave the assistant
- `AI-LOG.md`
- `SELF_ASSESSMENT_REPORT.md`

What `PROTOTYPE.md` must contain

1. **Team** — full name and student ID of every member
2. **How to open it** — "open `index.html`", and whether it needs a network connection (a framework loaded from a CDN does). With React: how to run it (`npm ci`, then `npx vite preview`)
3. **Screens** — one row per page: the file (React: the path), the GradStudio screen it comes from, what the user does there, and which page each of its links opens
4. **Your design choices** — the main ways your pages differ from the brief's screens, and why
5. **Not wired yet** — controls on these pages that lead outside the three screens (search, settings, log out) and are left for later milestones
6. **CSS** — the framework or plain CSS, your spacing scale, your breakpoints

What is scored

#	Criterion	Points
1	Three linked screens that trace back to the GradStudio brief	20
2	Responsive at 360, 768 and 1280 px	25
3	Semantic, valid HTML	20
4	Design you can check	10
5	The brief and the rules file	10
6	`AI-LOG.md`	10
7	Commit history	5

Criteria 1–3 are marked by **opening your pages**, not by reading the code: a browser script loads every page from the folder at 360, 768 and 1280 px with JavaScript turned off (React: served, with JavaScript on). It checks that every link and form `action` opens a file or route that exists, measures the horizontal scroll, looks for content cut off past the right edge, checks the viewport `<meta>`, measures buttons and links at 360 px, and lists text size, contrast, landmarks, headings, labels and `alt`. We run the W3C checker on every page ourselves. Criterion 4 adds a reading of your CSS (the spacing scale, `!important`, what you load), and criterion 1 a check of the features against the brief. Your `screenshots/` and `checks/` show what you saw; where they disagree with our run, our run counts.

1. Three linked screens — 20 points

Points	What it looks like
17–20	Three pages, in the order of the main flow, starting at `index.html`. Together they deliver the features the brief gives its three screens. Layout, content and colours are your team's choice: a difference from the brief's screens does not cost marks. Every main-flow link or button opens the right page; nothing on the main flow leads to `#` or to a missing file. None of the problems on the brief's *Fix these* list appears.
11–16	Three linked pages, but one main-flow link is broken or leads to `#`, or a feature the brief gives a screen is missing, or a problem on its *Fix these* list appears.
5–10	Fewer than three pages; or, for static pages, links work only on a server (`/page.html`); or the pages are not the GradStudio main flow and `PROTOTYPE.md` does not say why.
0–4	No pages, or a ready-made template or an existing site instead of the GradStudio main flow.

2. Responsive — 25 points

Points	What it looks like
22–25	On every page, at all three widths: no horizontal scroll, nothing cut off or overlapping, text readable without zooming (a table that scrolls inside its own wrapper at 360 px is fine — session 3). At 360 px, buttons and links that stand on their own are at least 44 px tall — a link inside a sentence is exempt — and the long item wraps instead of pushing the page wider. The layout changes where the screen allows it — for example one column at 360, more at 1280 — instead of the phone column stretched across a laptop. The nine screenshots match the pages.
15–21	Works at all three widths with one or two flaws: a small horizontal scroll, a tap target under 44 px, an image that overflows.
7–14	One width is broken on several pages — usually 360 px, and usually one of session 3's five: a fixed `width`, a table that cannot shrink, an image without `max-width: 100%`, tap targets under 44 px, a missing viewport `<meta>`.
0–6	Only one width works, or the pages do not open.

Check the horizontal scroll yourself: set DevTools' device toolbar to each width, then paste into the Console `document.documentElement.scrollWidth > document.documentElement.clientWidth` — it must print `false`. The marking script runs the same line, plus two checks the line cannot do:

- **Missing viewport `<meta>`.** A phone then lays the page out 980 px wide and shrinks it, and the line still prints `false`. `document.documentElement.clientWidth` must print the width you set.
- **Hidden overflow.** `overflow-x: hidden` on `html`, `body` or a wrapper hides the scroll bar, not the problem: the content is cut off and the line prints `false`. We count that as cut off. This line lists what sticks out past the right edge and must print `[]` (anything inside a table's scrolling wrapper is fine): `[...document.querySelectorAll('body *')].filter(e => e.getBoundingClientRect().right > document.documentElement.clientWidth)`

Tap targets: Bootstrap's default `.btn` is 38 px tall and its `.nav-link` 40 px; Tailwind's `py-2` on 16 px text is 40 px. Set a minimum height for your buttons and links rather than trusting the framework.

3. Semantic, valid HTML — 20 points

Points	What it looks like
17–20	Every page: **0 errors** in the W3C checker (validator.w3.org/nu; warnings allowed), with the result saved in `checks/`. `<header>`, `<nav>`, `<main>` and `<footer>` where they belong; one `<h1>` and no skipped heading level; an action is a `<button>`, going to another page is an `<a href>` (or a form's submit button with `action`); every input has a visible `<label>` — clicking the words puts the cursor in the box; every image has `alt`.
11–16	0 errors everywhere, but one or two of: a skipped heading level, a `<div>` used as a button, an input with only a placeholder, an image without `alt`.

Points	What it looks like
5–10	Checker errors on one or more pages, or pages built mostly from `<div>` with no landmarks.
0–4	Errors on most pages and no results in `checks/`.

Valid is not the same as semantic (session 4): an all-`<div>` page can show 0 errors. Check both — the W3C checker for grammar, DevTools → Elements → **Accessibility** for meaning.

4. Design you can check — 10 points

Points	What it looks like
9–10	One spacing scale used throughout — your own custom properties or the framework's scale — with no more than a handful of values outside it (in Tailwind, an arbitrary value such as `p-[13px]` is outside the scale). Body text 16 px or larger. Body text and buttons pass 4.5:1 contrast in the DevTools colour picker. One CSS framework at most, and no `!important` in your own CSS (the framework's file does not count). Each page has one thing that is clearly the most important.
5–8	Two of those slip: magic numbers here and there, a grey that fails contrast, a second framework loaded "just for the icons".
0–4	No visible system: gaps, sizes and colours guessed page by page.

5. The brief and the rules file — 10 points

`brief.md` is what you actually gave the assistant, not a summary written afterwards. Several prompts are fine — paste them in order. The rules file is the one at the root of your repository.

Points	What it looks like
9–10	The brief hands over the GradStudio screens and states the constraints an assistant breaks when nobody says them: your framework and spacing scale, mobile first and your breakpoints, no fixed widths, no new dependency and no JavaScript framework beyond React if you chose it, the semantic rules (landmarks, one `<h1>`, labels, buttons versus links), relative links or router links. It says how *done* is checked: three widths, the W3C checker, the Accessibility pane. The rules file records the stack and a *never* list for the whole team.
5–8	A real brief with two or three of those constraints left out; or a good brief and no rules file.
1–4	One line: "make these wireframes responsive".
0	No brief.

6. `AI-LOG.md` — 10 points

One entry per page or task you gave the assistant. The lines we read first are **Changed** and **Rejected**.

Points	What it looks like
9–10	Each entry names the tool and the request, and at least one entry records something the assistant got wrong, **which check caught it** (a width, the W3C checker, the Accessibility pane) and what you told it — e.g. "at 360 px the price table scrolled sideways; asked for a scrolling wrapper on the table only". Specific enough to find in the commits.
5–8	Honest, with tool and request, but *Changed/Rejected* say "none" everywhere or stay vague ("fixed some CSS").
1–4	One entry for the whole milestone, or clearly written after the fact.
0	Missing, or contradicted by the code or the history.

`Tool: none` is a valid entry for a task you did without an assistant. "Nothing went wrong" is acceptable — if it says which checks you ran.

7. Commit history — 5 points

Points	What it looks like
5	A first commit with the rules file, then at least one commit per page and one for the responsive pass, each message saying what changed. The log can be checked against the files.
2–4	Some history, but commits mix several pages or the messages say only "update".
0–1	One commit with everything, or no `.git` folder in the zip.

Commit under the name of whoever did the work — set `git config user.name` and `user.email` on each member's machine, since the zip carries those and not a GitHub account. When a team asks for marks to be split, the history is the first thing we read.

How to submit

One zip file per team, submitted by one member, named after the student IDs **and the total you give yourselves**: `<StudentID1>-<StudentID2>-<StudentID3>_<total>.zip`, IDs in ascending order — for example `24120001-24120042-24120117_75.zip`. A team of two has two IDs; a team of one, one.

The zip must contain **`SELF_ASSESSMENT_REPORT.md`**: one row per criterion in this rubric, the marks you claim, and evidence pointing at a file, a screenshot, a checker result or a commit — plus a short *what we did not manage*. The total in that table is the number in the file name. Template in *Course documents* on Classroom.

A wrong file name costs no marks but delays your result — the grading script matches submissions to the class list by that name.

Turn in from your `<StudentID>@student.hcmus.edu.vn` Google account.

Resubmit as often as you like before the deadline; the last version is graded. **Late:** under 2 hours −20% of the total · 2–24 hours −50% · after 24 hours not accepted.

Honesty adjustment

Your self-assessment is compared with the mark you actually earn. The gap is `your total − the mark`, on the same 100-point scale.

Gap	Adjustment
within ±10	none — this is normal calibration
+11 to +20	−3
+21 to +30	−6
more than +30	−10
−21 or worse	−3 — read the rubric before you score yourselves down
no `SELF_ASSESSMENT_REPORT.md`	−10, and the file-name total is ignored

A criterion you claim with no evidence line counts as claimed-and-not-done for this comparison. The adjustment never takes a submission below 0.

Converting to the course mark

PA#2 is 3% of the course: `mark / 100 × 3`. A team scoring 80 earns 2.4 of the 3 percentage points. Every member of the team receives the same mark, unless the team asks for the marks to be split (criterion 7).

Notes

- Same team as PA#1. A team that changes members tells the lecturer before it starts, not in the submission.
- Three screens done well beat six half-done. Extra pages earn no extra marks.
- Run the three widths with JavaScript off (React: on, through `npx vite preview`), the W3C checker and the two Console lines yourselves before you score criteria 1–4: the marking script runs the same checks, and the others listed under *What is scored*.
- PA#3 rebuilds these screens in React; if you already use React here, it continues from them. The layout, the outline and the links you settle now are what it starts from.
- You may be asked to explain any line in class — including lines your assistant wrote. Explaining *what a line does and how you checked it* is enough; nobody will ask you to write it from memory.